



SETUP GUIDE

Benchmarking Environment for Attention Experiments

Tools: PyTorch 2.0+ (with SDPA/Flash Attention), Hugging Face Transformers,
Memory Profiling Tools

Prerequisite: Python 3.9+ with pip, virtual environment (from Week 6)

SECTION 1: WSL (Windows Subsystem for Linux) — RECOMMENDED FOR WINDOWS

PREREQUISITES

- WSL2 with Ubuntu (from Week 4)
- Python 3.9+ (python3 --version)
- pip installed (pip3 --version)
- Virtual environment from Week 6 (or create new one)
- (Optional) NVIDIA GPU with CUDA for Flash Attention + GPU profiling

STEP 1: Activate or create virtual environment

Reuse Week 6 environment:
source ~/opt-env/bin/activate

Or create fresh:
python3 -m venv ~/bench-env
source ~/bench-env/bin/activate

STEP 2: Install PyTorch 2.0+ (required for SDPA/Flash Attention)

CPU-only:
pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu

GPU (CUDA 12.x):
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu121

GPU (CUDA 11.8):
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu118

Verify PyTorch version (must be 2.0+):
python -c "import torch; print('PyTorch:', torch.__version__); print('CUDA available:', torch.cuda.is_available())"



STEP 3: Hugging Face Transformers + dependencies

pip install transformers accelerate tokenizers sentencepiece

Verify:

```
python -c "from transformers import AutoModelForCausalLM, AutoTokenizer; print('Transformers OK')"
```

STEP 4: Install benchmarking and profiling tools

pip install numpy matplotlib psutil

For GPU memory profiling (if GPU available):

torch.cuda.memory_* functions are built into PyTorch — no extra install

For CPU memory profiling:

pip install memory-profiler

STEP 5: Download a small model for benchmarking

```
python3 << 'EOF'
```

```
from transformers import AutoModelForCausalLM, AutoTokenizer
```

GPT-2 small (124M params) — fits on any hardware

```
print("Downloading GPT-2...")
```

```
tokenizer = AutoTokenizer.from_pretrained("gpt2")
```

```
model = AutoModelForCausalLM.from_pretrained("gpt2")
```

```
print("✓ GPT-2 downloaded and cached")
```

Verify generation works

```
inputs = tokenizer("Hello, world", return_tensors="pt")
```

```
outputs = model.generate(**inputs, max_new_tokens=10)
```

```
print("✓ Generation test:", tokenizer.decode(outputs[0]))
```

```
EOF
```

STEP 6: Verify Flash Attention / SDPA availability

```
python3 << 'EOF'
```

```
import torch
```

```
import torch.nn.functional as F
```

```
print(f"PyTorch version: {torch.__version__}")
```

```
print(f"CUDA available: {torch.cuda.is_available()}")
```

Check SDPA exists (PyTorch 2.0+)

```
assert hasattr(F, 'scaled_dot_product_attention'), "SDPA not available — need PyTorch 2.0+"  
print("✓ scaled_dot_product_attention available")
```

```
if torch.cuda.is_available():  
    # Check Flash Attention backend  
    print(f" Flash SDP enabled: {torch.backends.cuda.flash_sdp_enabled()}")  
    print(f" Mem-efficient SDP: {torch.backends.cuda.mem_efficient_sdp_enabled()}")  
    print(f" Math SDP: {torch.backends.cuda.math_sdp_enabled()}")  
else:  
    print(" (CPU mode — math SDP backend will be used)")  
  
print("\n Environment ready for attention experiments!")  
EOF
```

STEP 7: Run Quick Benchmark Test

```
python3 << 'EOF'  
import torch  
import torch.nn.functional as F  
import time  
  
print("=" * 60)  
print(" ATTENTION BENCHMARK: Standard vs SDPA")  
print("=" * 60)  
  
device = "cuda" if torch.cuda.is_available() else "cpu"  
dtype = torch.float16 if torch.cuda.is_available() else torch.float32  
  
# Test parameters  
batch_size = 4  
num_heads = 8  
head_dim = 64  
seq_lengths = [128, 512, 1024]  
  
for seq_len in seq_lengths:  
    Q = torch.randn(batch_size, num_heads, seq_len, head_dim, device=device, dtype=dtype)  
    K = torch.randn(batch_size, num_heads, seq_len, head_dim, device=device, dtype=dtype)  
    V = torch.randn(batch_size, num_heads, seq_len, head_dim, device=device, dtype=dtype)  
  
    # Standard attention  
    def standard_attention(Q, K, V):  
        scale = head_dim ** -0.5  
        attn_weights = torch.matmul(Q, K.transpose(-2, -1)) * scale
```

```
attn_weights = torch.softmax(attn_weights, dim=-1)
return torch.matmul(attn_weights, V)

# Warm up
for _ in range(10):
    _ = standard_attention(Q, K, V)
    _ = F.scaled_dot_product_attention(Q, K, V)
if device == "cuda":
    torch.cuda.synchronize()

# Benchmark standard
start = time.perf_counter()
for _ in range(50):
    _ = standard_attention(Q, K, V)
if device == "cuda":
    torch.cuda.synchronize()
std_time = (time.perf_counter() - start) / 50

# Benchmark SDPA
start = time.perf_counter()
for _ in range(50):
    _ = F.scaled_dot_product_attention(Q, K, V)
if device == "cuda":
    torch.cuda.synchronize()
sdpa_time = (time.perf_counter() - start) / 50

speedup = std_time / sdpa_time
print(f" Seq={seq_len:>5} | Standard: {std_time*1000:.2f}ms | SDPA: {sdpa_time*1000:.2f}ms |
Speedup: {speedup:.2f}x")

print("\n Benchmark complete!")
print(" (Speedup grows with longer sequences — Flash Attention shines at 2K+)")
EOF
```

SECTION 2: macOS

PREREQUISITES

- macOS 12+ (Monterey or later)
- Python 3.9+ (brew install python@3.11)
- pip installed



STEP 1: Create/activate virtual environment

```
python3 -m venv ~/bench-env  
source ~/bench-env/bin/activate
```

STEP 2: Install PyTorch (MPS for Apple Silicon, CPU for Intel)

```
pip install torch torchvision
```

Verify

```
python -c "import torch; print('PyTorch:', torch.__version__); print('MPS available:',  
torch.backends.mps.is_available())"
```

Note: Flash Attention (GPU SRAM tiling) is NVIDIA-specific.

On Apple Silicon, PyTorch SDPA uses an optimized MPS backend.

On Intel Mac, CPU math backend is used.

STEP 3: Install Hugging Face + tools

```
pip install transformers accelerate tokenizers sentencepiece numpy matplotlib psutil
```

STEP 4: Download GPT-2 and run verification (same as WSL Step 5 + 6 + 7)

SECTION 3: LINUX (Ubuntu/Debian)

PREREQUISITES

- Ubuntu 20.04+ or Debian 11+
- Python 3.9+
- (Optional) NVIDIA GPU with CUDA 11.8+ for Flash Attention

STEP 1: Create virtual environment

```
python3 -m venv ~/bench-env  
source ~/bench-env/bin/activate
```

STEP 2: Install PyTorch 2.0+

CPU-only:

```
pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
```

GPU (CUDA 12.x):

```
pip install torch torchvision --index-url https://download.pytorch.org/whl/cu121
```

STEP 3: Install remaining tools



```
pip install transformers accelerate tokenizers sentencepiece numpy matplotlib psutil
```

STEP 4: Verify and run benchmarks (same as WSL Steps 5-7)

SECTION 4: ADDITIONAL BENCHMARK SCRIPTS

BENCHMARK A: KV Cache Memory Growth

```
python3 << 'EOF'
import torch
from transformers import AutoModelForCausalLM, AutoTokenizer

model_name = "gpt2"
tokenizer = AutoTokenizer.from_pretrained(model_name)
model = AutoModelForCausalLM.from_pretrained(model_name)
model.eval()

device = "cuda" if torch.cuda.is_available() else "cpu"
model = model.to(device)

prompt = "The future of artificial intelligence"
inputs = tokenizer(prompt, return_tensors="pt").to(device)

print("Token | Memory (MB) | Delta (MB)")
print("-" * 40)

def get_cache_size(past_kv):
    """Compute KV cache size — handles DynamicCache (new) and tuple (old) formats."""
    total = 0
    if hasattr(past_kv, 'layers'):
        for layer in past_kv.layers:
            k, v = layer.keys, layer.values
            if k is not None:
                total += k.nelement() * k.element_size()
            if v is not None:
                total += v.nelement() * v.element_size()
            elif hasattr(past_kv, 'key_cache'):
                for i in range(len(past_kv.key_cache)):
                    total += past_kv.key_cache[i].nelement() * past_kv.key_cache[i].element_size()
                    total += past_kv.value_cache[i].nelement() * past_kv.value_cache[i].element_size()
            else:
```

```
for k, v in past_kv:
    total += k.nelement() * k.element_size() + v.nelement() * v.element_size()
return total / 1024 / 1024

prev_mem = 0
with torch.no_grad():
    outputs = model(**inputs, use_cache=True)
    past_kv = outputs.past_key_values
    next_token = outputs.logits[:, -1:, :].argmax(dim=-1)

for i in range(30):
    if device == "cuda":
        mem = torch.cuda.memory_allocated() / 1024 / 1024
    else:
        mem = get_cache_size(past_kv)

    delta = mem - prev_mem if prev_mem > 0 else 0
    print(f" {i+1:>3} | {mem:>8.2f} | {delta:>+.2f}")
    prev_mem = mem

outputs = model(next_token, past_key_values=past_kv, use_cache=True)
past_kv = outputs.past_key_values
next_token = outputs.logits[:, -1:, :].argmax(dim=-1)

print("\n✓ KV cache grows linearly with each generated token")
EOF
```

BENCHMARK B: Assisted Generation vs Standard

```
python3 << 'EOF'
import torch
import time
from transformers import AutoModelForCausalLM, AutoTokenizer

print("Loading models...")
model_name = "gpt2" # Target model
tokenizer = AutoTokenizer.from_pretrained(model_name)
tokenizer.pad_token = tokenizer.eos_token

model = AutoModelForCausalLM.from_pretrained(model_name)
model.eval()

# For assisted generation, we use the same model as both
```

```
# (In practice, use a smaller model as assistant)
assistant_model = AutoModelForCausalLM.from_pretrained("distilgpt2")
assistant_model.eval()

device = "cuda" if torch.cuda.is_available() else "cpu"
model = model.to(device)
assistant_model = assistant_model.to(device)

prompt = "Machine learning models can be optimized for production by"
inputs = tokenizer(prompt, return_tensors="pt").to(device)
gen_length = 50

print(f"\nGenerating {gen_length} tokens...")
print("-" * 50)

# Standard generation
start = time.perf_counter()
with torch.no_grad():
    output_std = model.generate(**inputs, max_new_tokens=gen_length, do_sample=False)
std_time = time.perf_counter() - start

# Assisted generation
start = time.perf_counter()
with torch.no_grad():
    output_asst = model.generate(**inputs, max_new_tokens=gen_length,
                                assistant_model=assistant_model, do_sample=False)
asst_time = time.perf_counter() - start

print(f" Standard generation: {std_time:.3f}s")
print(f" Assisted generation: {asst_time:.3f}s")
print(f" Speedup: {std_time/asst_time:.2f}x")
print(f"\n Output (standard): {tokenizer.decode(output_std[0], skip_special_tokens=True)[:100]}...")
print(f"\n✓ Assisted generation complete")
print(f" Note: speedup increases with larger target models (70B) and better-matched assistants")
EOF
```

SECTION 5: TROUBLESHOOTING

PROBLEM: "PyTorch version is 1.x, need 2.0+ for SDPA"

CAUSE: Old PyTorch installed

FIX:



```
pip install --upgrade torch torchvision --index-url https://download.pytorch.org/whl/cpu
```

Or for GPU:

```
pip install --upgrade torch torchvision --index-url https://download.pytorch.org/whl/cu121
```

PROBLEM: "scaled_dot_product_attention not found"

CAUSE: PyTorch < 2.0

FIX: Upgrade PyTorch to 2.0+ (see above)

PROBLEM: "Flash SDP enabled: False" on NVIDIA GPU

CAUSE: GPU architecture too old (needs Ampere+) or CUDA version mismatch

FIX:

- Flash Attention requires SM80+ (A100, RTX 3090, etc.)
- Older GPUs (V100, T4) use mem_efficient_sdp instead — still faster than standard
- Ensure matching CUDA version: nvcc --version should match PyTorch CUDA build

PROBLEM: "CUDA out of memory" during benchmarks

CAUSE: Model or benchmark using too much GPU memory

FIX:

- Reduce batch size in benchmark scripts
- Use smaller model (GPT-2 instead of larger)
- Clear cache: torch.cuda.empty_cache()
- Reduce sequence length in tests

PROBLEM: "transformers.AutoModelForCausalLM — model not found"

CAUSE: Model not downloaded or network issue

FIX:

- Check internet connection
- Try: huggingface-cli login (if gated model)
- Use offline: set HF_HOME and pre-download

PROBLEM: Assisted generation slower than standard

CAUSE: Model too small for speculative benefit, or assistant too large

FIX:

- Expected with small models (GPT-2). Real benefit shows with 7B+ targets
- Ensure assistant model is significantly smaller than target
- The demo shows the mechanism; real speedup needs large target + small assistant

PROBLEM: Memory profiling shows no growth (CPU mode)

CAUSE: CPU memory management differs from GPU

FIX:

- Use memory_profiler package for CPU: @profile decorator
- Or track KV cache tensor sizes manually (as in Benchmark A script)



- GPU profiling (torch.cuda.memory_*) gives most precise results

SECTION 6: VERIFICATION CHECKLIST

[] python -c "import torch; print(torch.__version__)"

→ Version 2.0+ displayed

[] python -c "import torch.nn.functional as F; print(hasattr(F, 'scaled_dot_product_attention'))"

→ True

[] python -c "from transformers import AutoModelForCausalLM; print('OK')"

→ OK (no import errors)

[] Download GPT-2 model successfully

→ "GPT-2 downloaded and cached"

[] Generate text with GPT-2

→ Text generated without errors

[] Run attention benchmark (Step 7)

→ SDPA shows speedup over standard attention

[] (GPU) Flash SDP enabled

→ True on Ampere+ GPUs

[] (GPU) torch.cuda.memory_allocated() returns values

→ Non-zero after model load

[] Run KV cache growth benchmark

→ Memory grows linearly per token

[] Run assisted generation benchmark

→ Both modes complete without error

ALL CHECKS PASSED = Environment ready for Week 7 experiments